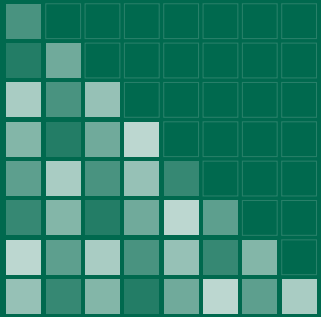
The decoder-only Transformer

The Maps Behind LLM

AMSOU Seminar · Department of Mathematics

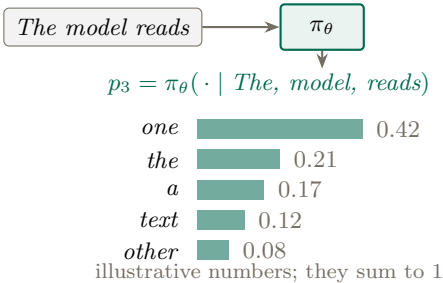


$$A = \mathrm{softmax}_{\mathrm{row}}\left(QK^{\top}/\sqrt{d_k} + M\right)$$

The *model* *reads* *one* ?

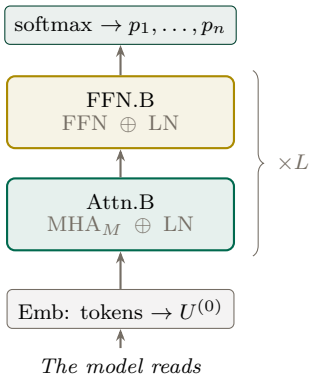
$$p_i = \pi_\theta(\cdot \mid y_{\le i})$$

What a language model computes

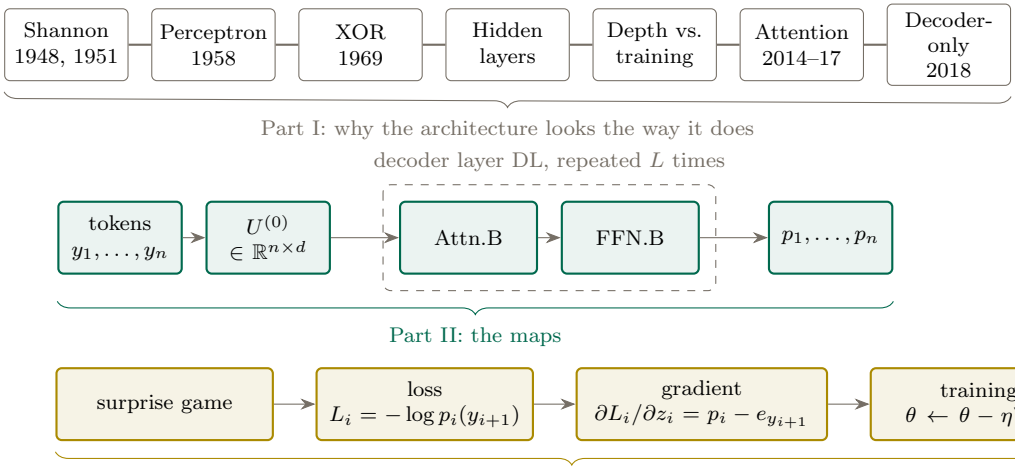


Generation. $y_{n+1} \sim p_n$, append, repeat.

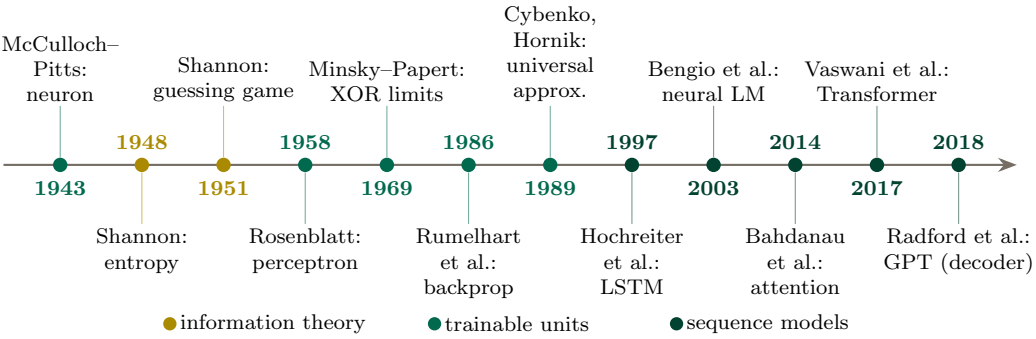
inside π_θ



The road map


Shannon 1948, 1951
Perceptron 1958
XOR 1969
Hidden layers
Depth vs. training
Attention 2014–17
Decoder-only 2018
Part I: why the architecture looks the way it does
decoder layer DL, repeated L times
tokens $y_1, \ldots, y_n$
$U^{(0)} \in \mathbb{R}^{n \times d}$
Attn.B
FFN.B
$p_1, \ldots, p_n$
Part II: the maps
surprise game
loss $L_i = -\log p_i(y_{i+1})$
gradient $\partial L_i / \partial z_i = p_i - e_{y_{i+1}}$
training $\theta \leftarrow \theta - \eta \nabla_\theta \ell$
Part III: the loss and its gradients

A short history



Three threads meet in 2017: Shannon's *next-symbol* objective, Rosenblatt's *trainable units*, and *attention*.

Shannon (1948, 1951): language as a stochastic process

Entropy rate. $(Y_t)_{t\geq 1}$ stationary on a finite alphabet $\mathcal{V}$:

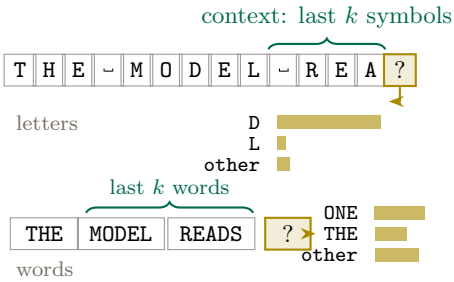
$$H_\infty = \lim_{n\to\infty} H(Y_n \mid Y_1, \ldots, Y_{n-1}), \qquad H(Y_n \mid Y_{<n}) = -\mathbb{E}\log_2 \mathbb{P}(Y_n \mid Y_{<n}).$$

1948: k-gram (Y_t letters, W_t words)

$$\mathbb{P}(Y_n \mid Y_{<n}) \approx \mathbb{P}(Y_n \mid Y_{n-k}, \ldots, Y_{n-1})$$
$$\mathbb{P}(W_n \mid W_{<n}) \approx \mathbb{P}(W_n \mid W_{n-k}, \ldots, W_{n-1})$$

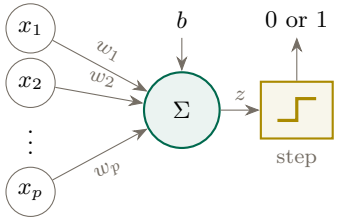
1951: English

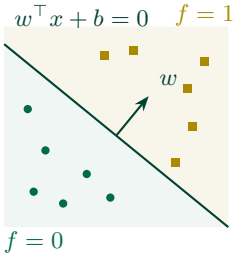
$0.6 \lesssim H_\infty \lesssim 1.3$ bits/letter



LLM loss (cross-entropy): $L_i = -\log p_i(y_{i+1})$.

Rosenblatt (1958): the perceptron





Trained: $(w, b) \in \mathbb{R}^p \times \mathbb{R}$.

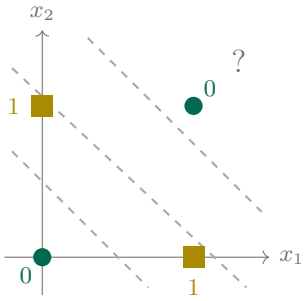
$$f_{w,b} : \mathbb{R}^p \to \{0, 1\}$$
$$f_{w,b}(x) = \mathbf{1}\{w^\top x + b > 0\}$$

Learning rule. Data (x, t), $t \in \{0, 1\}$, step $\eta > 0$:

$$(w, b) \leftarrow (w, b) + \eta \left(t - f_{w,b}(x)\right)(x, 1)$$

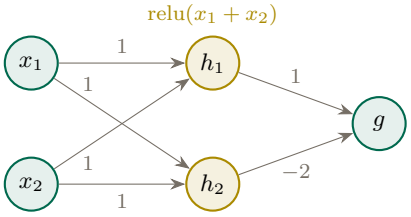
Minsky–Papert (1969): XOR defeats one line

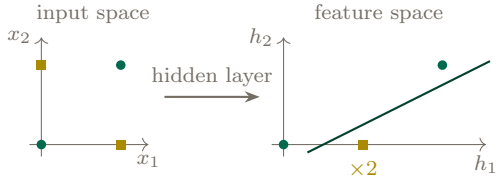
x_1	x_2	$x_1 \oplus x_2$
0	0	0
1	0	1
0	1	1
1	1	0



Proposition. There is no $(w, b) \in \mathbb{R}^2 \times \mathbb{R}$ with $f_{w,b}(x) = x_1 \oplus x_2$ for all $x \in \{0, 1\}^2$.

A hidden layer solves XOR





$\text{relu}(z) = \max(z, 0)$

x	$s = x_1 + x_2$	$h_1 = s$	$h_2 = \text{relu}(s-1)$	$g = h_1 - 2h_2$
00	0	0	0	0
10	1	1	0	1
01	1	1	0	1
11	2	2	1	**0**

$g(x) = w_2^\top \text{relu}(W_1 x + b_1) + b_2$: affine $\to$ activation $\to$ affine = the FFN of every decoder layer.

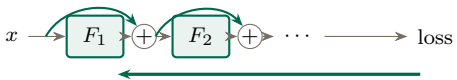
Expressivity vs. trainability

Deep plain stack

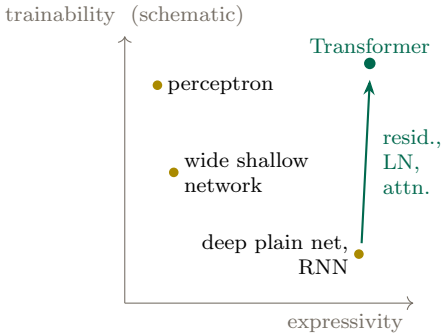


$$\frac{\partial \ell}{\partial \theta_1} = \frac{\partial \ell}{\partial h^{(L)}} J_L \cdots J_2 \frac{\partial F_1}{\partial \theta_1} \to 0$$

With residual connections



$J_l = I + \partial G_l$: identity path survives

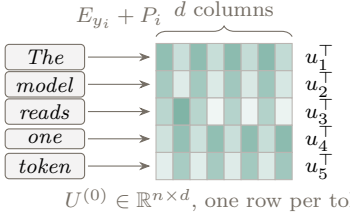


Expressivity: functions the network can represent (↑ with L). **Trainability:** gradient descent finds good θ.
$L \uparrow \Rightarrow$ longer product $J_L \cdots J_{l+1}$, $J_l = \partial F_l / \partial h^{(l-1)} \Rightarrow \partial \ell / \partial \theta_l \to 0 \Rightarrow$ trainability ↓. Fix: residual $\oplus$ + LN.

Emb, step by step: from tokens to $U^{(0)}$

1. **Tokenise.** Text $\to$ tokens with ids in $\mathcal{V} = \{1, \dots, |\mathcal{V}|\}$. Toy model: characters ($|\mathcal{V}| = 37$); LLMs: subword pieces ($|\mathcal{V}| \sim 10^4$–10^5).
2. **One-hot.** $y_i \;\mapsto\; e_{y_i} \in \{0,1\}^{|\mathcal{V}|}$, with a 1 at entry y_i.
3. **Look up.** $E_{y_i} = E\, e_{y_i}$ = column y_i of $E \in \mathbb{R}^{d \times |\mathcal{V}|}$ (in code: a table lookup).
4. **Add the position.** $u_i = E_{y_i} + P_i$, $P_i = P\, e_i$, $e_i \in \{0,1\}^{n_{\text{ctx}}}$.
5. **Stack.** Row i of $U^{(0)} \in \mathbb{R}^{n \times d}$ is $u_i^\top$.

$$U^{(0)} = \text{Emb}(y_1, \dots, y_n) = \begin{pmatrix} (E_{y_1} + P_1)^\top \\ \vdots \\ (E_{y_n} + P_n)^\top \end{pmatrix}.$$



Positions. Trained $P \in \mathbb{R}^{d \times n_{\text{ctx}}}$:

$$P = \left[\, P_1 \,\middle|\, \cdots \,\middle|\, P_{n_{\text{ctx}}} \right]$$

$$P_i = P\, e_i \in \mathbb{R}^d, \quad e_i \in \{0,1\}^{n_{\text{ctx}}}$$

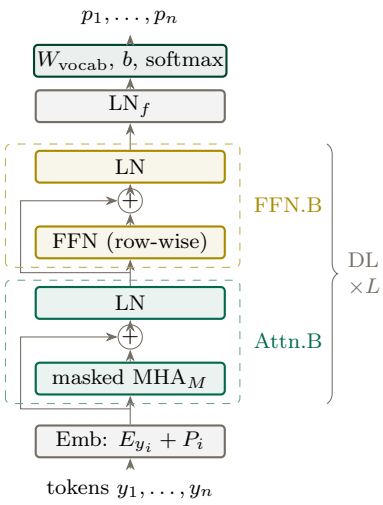
P_i = learned vector of position i; entries start $\sim \mathcal{N}(0, 0.1^2)$.

The decoder-only architecture

Each map below is $\mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$:

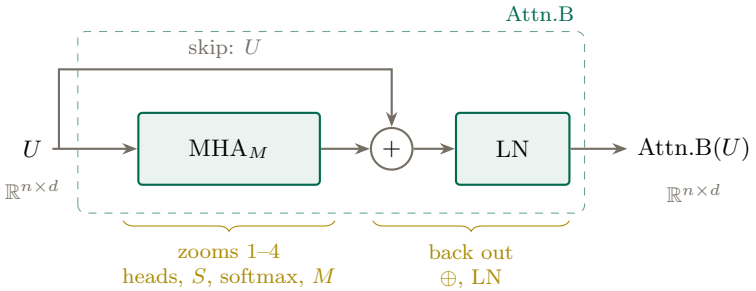
$$\begin{aligned} \mathrm{DL}(U) &= \mathrm{FFN.B}\left(\mathrm{Attn.B}(U)\right) \\ \mathrm{Attn.B}(U) &= \mathrm{LN}\left(U + \mathrm{MHA}_M(U)\right) \\ \mathrm{FFN.B}(U) &= \mathrm{LN}\left(U + \mathrm{FFN}(U)\right) \end{aligned}$$

$\mathrm{DL}_1, \ldots, \mathrm{DL}_L$: same form, separate parameters.



Attn.B: the block

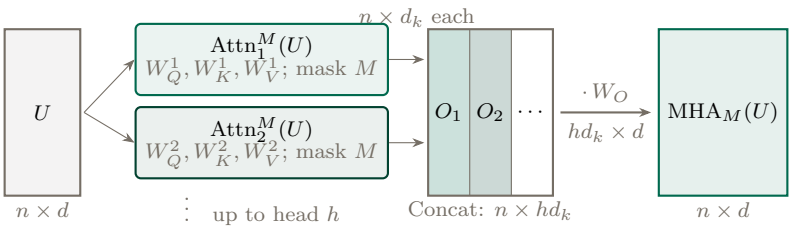
$$\mathrm{Attn.B} : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}, \qquad \mathrm{Attn.B}(U) = \mathrm{LN}\left(U + \mathrm{MHA}_M(U)\right)$$



Zoom 1: MHA_M = heads + concat

h heads, $d_k = d/h$. Trained: $W_Q^r, W_K^r, W_V^r \in \mathbb{R}^{d\times d_k}$ $(r = 1, \dots, h)$ and $W_O \in \mathbb{R}^{hd_k \times d}$. Fixed: the causal mask M. The map $\mathrm{MHA}_M : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$ is

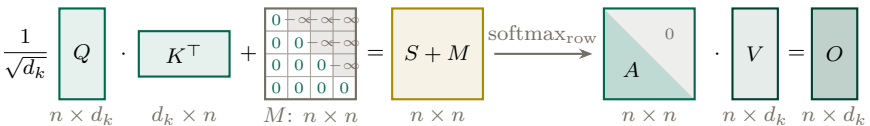
$$\mathrm{MHA}_M(U) = \underbrace{\mathrm{Concat}\left(\mathrm{Attn}_1^M(U), \dots, \mathrm{Attn}_h^M(U)\right)}_{\in \mathbb{R}^{n\times hd_k}} W_O.$$



$hd_k = d \;\Rightarrow\; \mathrm{MHA}_M(U) \in \mathbb{R}^{n\times d}$.

Head $r \in \{1, \dots, h\}$. Trained: $W_Q^r, W_K^r, W_V^r \in \mathbb{R}^{d \times d_k}$, giving $Q = UW_Q^r$, $K = UW_K^r$, $V = UW_V^r \in \mathbb{R}^{n \times d_k}$. Fixed: the mask $M \in \{0, -\infty\}^{n \times n}$.

$$\mathrm{Attn}_r^M : \mathbb{R}^{n \times d} \to \mathbb{R}^{n \times d_k}, \qquad \mathrm{Attn}_r^M(U) = \underbrace{\mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)}_{A \in \mathbb{R}^{n \times n}} V$$



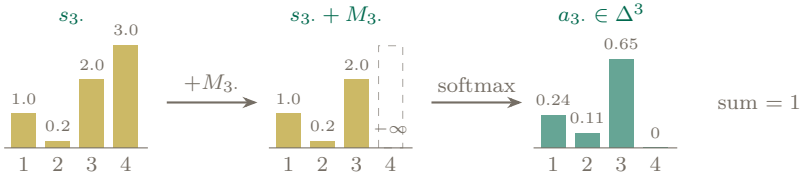
Row view. $o_i = \sum_j a_{ij} v_j$, $a_{i\cdot} \in \Delta^{n-1}$, so $o_i \in \mathrm{conv}\{v_j : M_{ij} = 0\} = \mathrm{conv}\{v_1, \dots, v_i\}$, where $\Delta^{n-1} = \left\{a \in \mathbb{R}^n : a_j \geq 0 \text{ for all } j, \sum_{j=1}^n a_j = 1\right\} \subset \mathbb{R}^n$.

Statistician's reading: Nadaraya–Watson smoothing of the v_j with kernel $\kappa(q_i, k_j) = \exp\left(q_i^\top k_j / \sqrt{d_k}\right)$.

Zoom 3: scores and softmax

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{n\times n}, \quad s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}; \qquad a_{i\cdot} = \mathrm{softmax}\left(s_{i\cdot} + M_{i\cdot}\right) \in \Delta^{n-1} \subset \mathbb{R}^n$$

$$\mathrm{softmax} : \mathbb{R}^n \to \Delta^{n-1}, \qquad \mathrm{softmax}(z)_j = \frac{e^{z_j}}{\sum_k e^{z_k}} \in \mathbb{R}$$



$\mathrm{softmax}(z + c\mathbf{1}) = \mathrm{softmax}(z); \quad z_j = -\infty \Rightarrow \mathrm{softmax}(z)_j = 0.$

Zoom 4: why the mask M

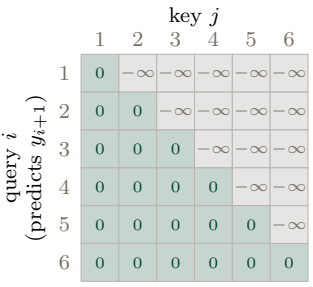
Training window: $y = (y_1, \ldots, y_{n+1})$, input $y_{1:n}$, and position i is scored by $L_i = -\log p_i[y_{i+1}]$.

Without a mask, row i of MHA (no mask) sees $u_{i+1} = E_{y_{i+1}} + P_{i+1}$, which *contains the answer*. A minimiser can copy it, so $L_i \to 0$ and nothing about language is learned.

Causal mask. $M \in \{0, -\infty\}^{n \times n}$ with $M_{ij} = 0$ for $j \le i$ and $M_{ij} = -\infty$ for $j > i$. With $s_{ij} = q_i^\top k_j / \sqrt{d_k}$:

$$a_{ij} = \frac{e^{s_{ij}}}{\sum_{j' \le i} e^{s_{ij'}}} \quad (j \le i),$$

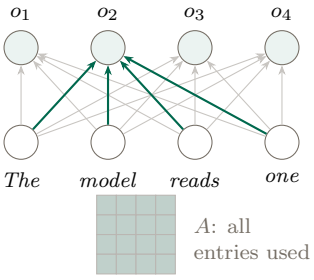
and $a_{ij} = 0$ for $j > i$.



M is part of π_θ: used in training and at inference.

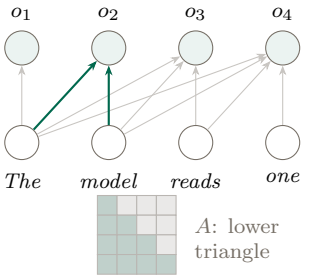
Zoom 4: Attn vs. Attn^M

$$\mathrm{Attn}_r(U) = \mathrm{softmax}_{\mathrm{row}}\left(QK^\top/\sqrt{d_k}\right)V$$



Every token sees every token (*encoders*).

$$\mathrm{Attn}_r^M(U) = \mathrm{softmax}_{\mathrm{row}}\left(QK^\top/\sqrt{d_k} + M\right)V$$



Token i sees tokens $1, \ldots, i$ (*decoders*).

Decoder-only: no encoder, hence no cross-attention.

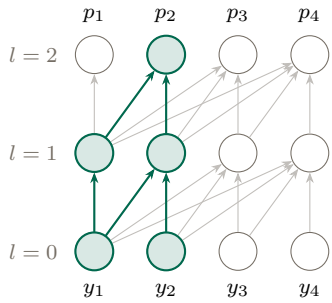
Zoom 4: mask $\Rightarrow p_i = \pi_\theta(\cdot \mid y_{\le i})$

Claim. $p_i = \pi_\theta(\cdot \mid y_{\le i})$: p_i is a function of $y_{\le i} = (y_1, \ldots, y_i)$ only, so $p_i(v) = \mathbb{P}_\theta(y_{i+1} = v \mid y_{\le i})$.

Proposition. With the causal mask M, for every layer l and position i, row i of $U^{(l)}$ depends only on $y_1, \ldots, y_i$. Hence the claim holds.

Proof sketch (induction on l). Row i of $U^{(0)}$ is $E_{y_i} + P_i$. In each masked head Attn$_r^M$, $a_{ij} = 0$ for $j > i$, so row i mixes rows $j \le i$ only; the $\oplus$, LN and FFN act on each row separately. ■

Payoff. $\sum_{i=1}^{n} L_i = -\log \pi_\theta(y_{2:n+1} \mid y_1)$.



Green: dependencies of p_2.

Back out: residual $\oplus$ and LN

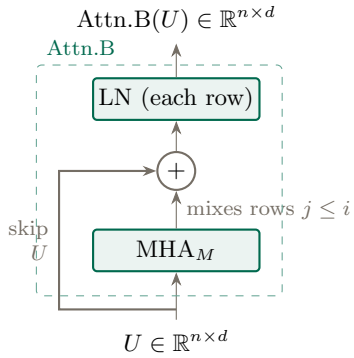
LayerNorm. Trained: $\gamma, \beta \in \mathbb{R}^d$. Fixed: a small $\varepsilon > 0$. For $u \in \mathbb{R}^d$ let $\mu(u) = \frac{1}{d}\mathbf{1}^\top u$ and $s^2(u) = \frac{1}{d}\|u - \mu(u)\mathbf{1}\|^2$. Then $\mathrm{LN} : \mathbb{R}^d \to \mathbb{R}^d$ is

$$\mathrm{LN}(u) = \gamma \odot \frac{u - \mu(u)\mathbf{1}}{\sqrt{s^2(u) + \varepsilon}} + \beta,$$

applied to each row of a matrix.

Block. $\mathrm{Attn.B} : \mathbb{R}^{n \times d} \to \mathbb{R}^{n \times d}$,

$$\mathrm{Attn.B}(U) = \mathrm{LN}\left(U + \mathrm{MHA}_M(U)\right).$$



Row i: rows $j \le i$ enter only via MHA_M; $\oplus$ and LN act row-wise.

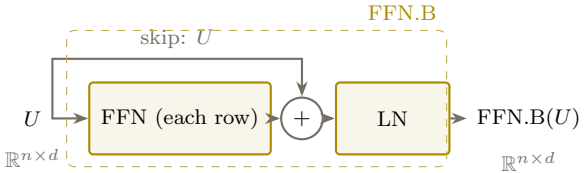
FFN.B: the block

Universal approximation (Cybenko 1989; Leshno et al. 1993). Let $\sigma : \mathbb{R} \to \mathbb{R}$ be continuous and not a polynomial, and let $K \subset \mathbb{R}^p$ be compact. Then the set of one-hidden-layer networks

$$\mathcal{N}_\sigma = \Big\{ g(x) = \sum_{m=1}^{M} c_m \, \sigma\big(a_m^\top x + b_m\big) \;:\; M \in \mathbb{N},\; a_m \in \mathbb{R}^p,\; b_m, c_m \in \mathbb{R} \Big\}$$

is dense in $C(K)$ in the sup norm. That is: for every continuous $f : K \to \mathbb{R}$ and every $\varepsilon > 0$ there is a $g \in \mathcal{N}_\sigma$ with $\sup_{x \in K} |f(x) - g(x)| < \varepsilon$.

$$\mathrm{FFN.B} : \mathbb{R}^{n \times d} \to \mathbb{R}^{n \times d}, \qquad \mathrm{FFN.B}(U) = \mathrm{LN}\big(U + \mathrm{FFN}(U)\big)$$



Zoom 1: the row-wise lift

U with rows $u_1^\top, \dots, u_n^\top$; for one row $u \in \mathbb{R}^d$:

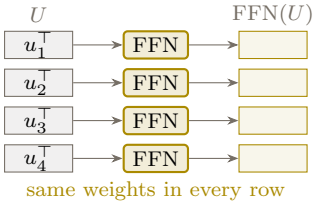
$$\mathrm{FFN}(u) = W_2\,\sigma(W_1 u + b_1) + b_2 \in \mathbb{R}^d$$

Stacked over the rows:

$$\mathrm{FFN}(U) = \begin{pmatrix} \mathrm{FFN}(u_1)^\top \\ \vdots \\ \mathrm{FFN}(u_n)^\top \end{pmatrix} \in \mathbb{R}^{n \times d}$$

Matrix form:

$$\mathrm{FFN}(U) = \sigma\big(U W_1^\top + \mathbf{1}_n b_1^\top\big) W_2^\top + \mathbf{1}_n b_2^\top$$



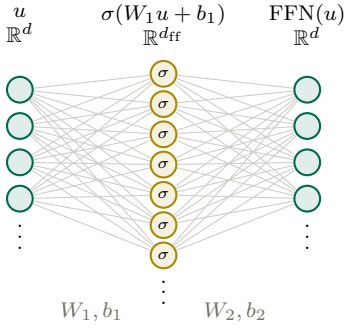
Zoom 2: the map FFN(u)

Trained: $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$, $b_1 \in \mathbb{R}^{d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$, $b_2 \in \mathbb{R}^d$ (typically $d_{\text{ff}} = 4d$). Fixed: an activation $\sigma : \mathbb{R} \to \mathbb{R}$, applied entrywise. The map $\text{FFN} : \mathbb{R}^d \to \mathbb{R}^d$ is

$$\text{FFN}(u) = W_2\, \sigma(W_1 u + b_1) + b_2.$$

Shapes.

$$u \in \mathbb{R}^d \xrightarrow{W_1 \cdot + b_1} \mathbb{R}^{d_{\text{ff}}} \xrightarrow{\sigma} \mathbb{R}^{d_{\text{ff}}} \xrightarrow{W_2 \cdot + b_2} \mathbb{R}^d.$$



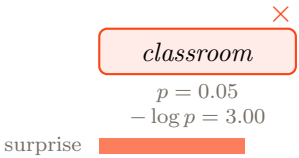
A game: how surprised are you?

"I am working hard on my physique, so you will find me at the _____."

surprise

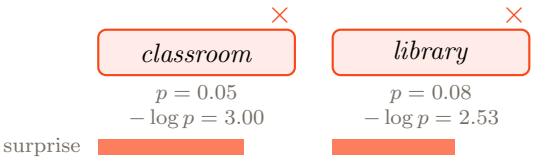
A game: how surprised are you?

"I am working hard on my physique, so you will find me at the _____."



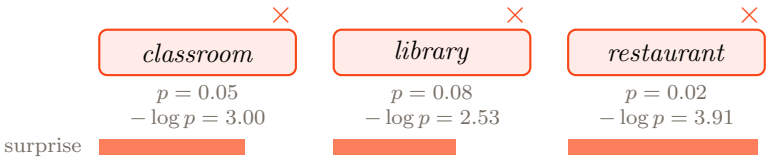
A game: how surprised are you?

"I am working hard on my physique, so you will find me at the _____."



A game: how surprised are you?

"I am working hard on my physique, so you will find me at the _____."



A game: how surprised are you?

"I am working hard on my physique, so you will find me at the _____."



A game: how surprised are you?

"I am working hard on my physique, so you will find me at the _____."



Next: the loss. Its goal. The loss trains the model to assign, at each position i (token index), a probability $p_i(v)$ to every token $v \in \mathcal{V}$ that matches how likely v is to come next given $y_1, \ldots, y_i$. *Choosing the next token from p_i (argmax or sampling) is a separate decoding step.*

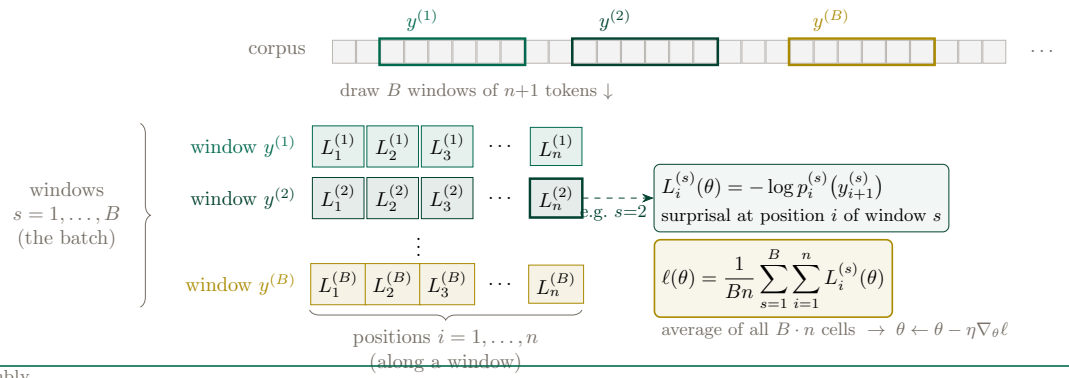
$L_i(\theta)$ is the surprisal of the true next token. At parameters θ, the model reads the prefix $y_1, \ldots, y_i$ and assigns probability $p_i(y_{i+1})$ to the token that actually comes next. $L_i(\theta) = -\log p_i(y_{i+1})$ measures how surprised it was: near 0 when the model was confident and right, large when it gave the true token little probability.

The loss

$$H = \mathrm{LN}_f\left(U^{(L)}\right) \in \mathbb{R}^{n\times d}, \quad h_i^\top = \text{row } i \text{ of } H \quad \text{(final hidden vector at position } i\text{)}.$$

$$z_i = W_{\text{vocab}}^\top h_i + b \in \mathbb{R}^{|\mathcal{V}|}, \qquad p_i = \mathrm{softmax}(z_i) \in \Delta(\mathcal{V}),$$

$$L_i(\theta) = -\log p_i(y_{i+1}) = -z_{i,y_{i+1}} + \log \textstyle\sum_{v\in\mathcal{V}} e^{z_{i,v}}, \quad \ell(\theta) = \frac{1}{Bn}\sum_{s=1}^{B}\sum_{i=1}^{n} L_i^{(s)}(\theta).$$



The derivative of the loss

Proposition (predicted − observed)**.**

$$\frac{\partial L_i}{\partial z_i} = p_i - e_{y_{i+1}} \in \mathbb{R}^{|\mathcal{V}|}$$

Proof. $\frac{\partial}{\partial z_{i,v}} \log \sum_u e^{z_{i,u}} = p_i(v)$ and $\frac{\partial z_{i,y_{i+1}}}{\partial z_{i,v}} = \mathbf{1}\{v = y_{i+1}\}$. ■

Chain rule.

$$\begin{aligned}
\frac{\partial L_i}{\partial b} &= p_i - e_{y_{i+1}}, \\
\frac{\partial L_i}{\partial W_{\text{vocab}}} &= h_i \left(p_i - e_{y_{i+1}}\right)^\top, \\
\frac{\partial L_i}{\partial h_i} &= W_{\text{vocab}} \left(p_i - e_{y_{i+1}}\right), \\
\nabla_\theta \ell &= \frac{1}{Bn} \sum_{s,i} \frac{\partial L_i^{(s)}}{\partial z_i^{(s)}} \frac{\partial z_i^{(s)}}{\partial \theta} \\
&= \frac{1}{Bn} \sum_{s,i} \left(p_i^{(s)} - e_{y_{i+1}^{(s)}}\right)^\top \frac{\partial z_i^{(s)}}{\partial \theta}.
\end{aligned}$$

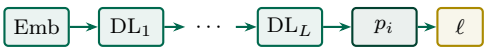
Training: gradient descent on ℓ

Input: token corpus; θ_0 random.

Toy run: $T = 1000$, $B = 8$, $n = 16$.

Trained:
$\theta = (E, P,$ all $W_\bullet, b_\bullet, \gamma_\bullet, \beta_\bullet)$.
Fixed: $M, \sigma, d, h, L, n, \eta, T, B$.

forward pass: compute and store each map's output



backward pass: chain rule, from ℓ back to θ

starts from $p_i - e_{y_{i+1}}$

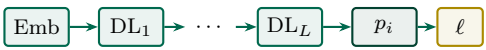
Training: gradient descent on ℓ

Input: token corpus; θ_0 random.
for $k = 0, \ldots, T-1$:

Toy run: $T = 1000$, $B = 8$, $n = 16$.

Trained:
$\theta = (E, P,$ all $W_\bullet, b_\bullet, \gamma_\bullet, \beta_\bullet)$.
Fixed: $M, \sigma, d, h, L, n, \eta, T, B$.

forward pass: compute and store each map's output



backward pass: chain rule, from ℓ back to θ

starts from $p_i - e_{y_{i+1}}$

Training: gradient descent on ℓ

Input: token corpus; θ_0 random.
for $k = 0, \ldots, T-1$:
draw $y^{(1)}, \ldots, y^{(B)} \in \mathcal{V}^{n+1}$

Toy run: $T = 1000$, $B = 8$, $n = 16$.

Trained:
$\theta = (E, P,$ all $W_\bullet, b_\bullet, \gamma_\bullet, \beta_\bullet)$.
Fixed: $M, \sigma, d, h, L, n, \eta, T, B$.

forward pass: compute and store each map's output



backward pass: chain rule, from ℓ back to θ
starts from $p_i - e_{y_{i+1}}$

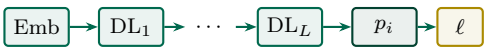
Training: gradient descent on ℓ

Input: token corpus; θ_0 random.
for $k = 0, \ldots, T-1$:
  draw $y^{(1)}, \ldots, y^{(B)} \in \mathcal{V}^{n+1}$
  forward pass: $\ell(\theta_k)$

Toy run: $T = 1000$, $B = 8$, $n = 16$.

Trained:
$\theta = (E, P,$ all $W_\bullet, b_\bullet, \gamma_\bullet, \beta_\bullet)$.
Fixed: $M, \sigma, d, h, L, n, \eta, T, B$.

forward pass: compute and store each map's output



backward pass: chain rule, from ℓ back to θ
starts from $p_i - e_{y_{i+1}}$

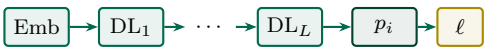
Training: gradient descent on ℓ

Input: token corpus; θ_0 random.
for $k = 0, \ldots, T-1$:
  draw $y^{(1)}, \ldots, y^{(B)} \in \mathcal{V}^{n+1}$
  forward pass: $\ell(\theta_k)$
  backward pass: $\nabla_\theta \ell(\theta_k)$
  $\theta_{k+1} = \theta_k - \eta \nabla_\theta \ell(\theta_k)$

Toy run: $T = 1000$, $B = 8$, $n = 16$.

Trained:
$\theta = (E, P,$ all $W_\bullet, b_\bullet, \gamma_\bullet, \beta_\bullet)$.
Fixed: $M, \sigma, d, h, L, n, \eta, T, B$.

forward pass: compute and store each map's output



backward pass: chain rule, from ℓ back to θ
starts from $p_i - e_{y_{i+1}}$

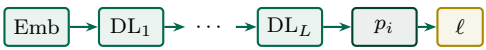
Training: gradient descent on ℓ

Input: token corpus; θ_0 random.
for $k = 0, \ldots, T-1$:
  draw $y^{(1)}, \ldots, y^{(B)} \in \mathcal{V}^{n+1}$
  forward pass: $\ell(\theta_k)$
  backward pass: $\nabla_\theta \ell(\theta_k)$
  $\theta_{k+1} = \theta_k - \eta \nabla_\theta \ell(\theta_k)$
return θ_T; freeze it.

Toy run: $T = 1000$, $B = 8$, $n = 16$.

Trained:
$\theta = (E, P,$ all $W_\bullet, b_\bullet, \gamma_\bullet, \beta_\bullet)$.
Fixed: $M, \sigma, d, h, L, n, \eta, T, B$.

forward pass: compute and store each map's output



backward pass: chain rule, from ℓ back to θ
starts from $p_i - e_{y_{i+1}}$

The whole map, and three take-aways

$$\mathcal{V}^n \xrightarrow{\text{Emb}} \mathbb{R}^{n\times d} \xrightarrow{\mathrm{DL}_L \circ \cdots \circ \mathrm{DL}_1} \mathbb{R}^{n\times d} \xrightarrow{\mathrm{LN}_f,\ W_{\text{vocab}},\ b} \mathbb{R}^{n\times|\mathcal{V}|} \xrightarrow{\text{softmax}_{\text{row}}} \Delta(\mathcal{V})^n$$

$$\mathrm{DL}_l = \mathrm{FFN.B}_l \circ \mathrm{Attn.B}_l, \qquad l = 1, \ldots, L.$$

1. Objective.
$\min_\theta \ell(\theta)$,
$\partial L_i / \partial z_i = p_i - e_{y_{i+1}}$.

2. Architecture.
$\mathrm{DL} = \mathrm{FFN.B} \circ \mathrm{Attn.B}$;
skips $\oplus$ and LN.

3. Mask.
$p_i = \pi_\theta(\cdot \mid y_{\le i})$.

The whole map, and three take-aways

$$\mathcal{V}^n \xrightarrow{\text{Emb}} \mathbb{R}^{n\times d} \xrightarrow{\mathrm{DL}_L \circ \cdots \circ \mathrm{DL}_1} \mathbb{R}^{n\times d} \xrightarrow{\mathrm{LN}_f,\ W_{\text{vocab}},\ b} \mathbb{R}^{n\times|\mathcal{V}|} \xrightarrow{\text{softmax}_{\text{row}}} \Delta(\mathcal{V})^n$$

$$\mathrm{DL}_l = \mathrm{FFN.B}_l \circ \mathrm{Attn.B}_l, \qquad l = 1, \ldots, L.$$

1. Objective.
$\min_\theta \ell(\theta)$,
$\partial L_i / \partial z_i = p_i - e_{y_{i+1}}$.

2. Architecture.
$\mathrm{DL} = \mathrm{FFN.B} \circ \mathrm{Attn.B}$;
skips $\oplus$ and LN.

3. Mask.
$p_i = \pi_\theta(\cdot \mid y_{\le i})$.

Thank *you* *for* *your* ?

The whole map, and three take-aways

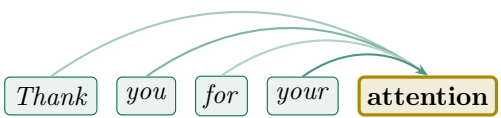
$$\mathcal{V}^n \xrightarrow{\text{Emb}} \mathbb{R}^{n\times d} \xrightarrow{\mathrm{DL}_L \circ \cdots \circ \mathrm{DL}_1} \mathbb{R}^{n\times d} \xrightarrow{\mathrm{LN}_f,\ W_{\text{vocab}},\ b} \mathbb{R}^{n\times|\mathcal{V}|} \xrightarrow{\text{softmax}_{\text{row}}} \Delta(\mathcal{V})^n$$

$$\mathrm{DL}_l = \mathrm{FFN.B}_l \circ \mathrm{Attn.B}_l, \qquad l = 1, \ldots, L.$$

1. Objective.
$\min_\theta \ell(\theta)$,
$\partial L_i / \partial z_i = p_i - e_{y_{i+1}}$.

2. Architecture.
$\mathrm{DL} = \mathrm{FFN.B} \circ \mathrm{Attn.B}$;
skips $\oplus$ and LN.

3. Mask.
$p_i = \pi_\theta(\cdot \mid y_{\le i})$.



$p(\textit{attention} \mid \textit{Thank you for your}) = 0.99$
$-\log p = 0.01$: no surprise. **Questions?**

References

- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal* 27. (See also Shannon 1951, Prediction and entropy of printed English.)
- Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review* 65.
- Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*.
- Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. OpenAI.